Job Prep
BY KRISHNAIK

Submissions open

SmartReco Build Challenge 2026

Powered by **Mesh API**

Build a behavioral AI recommendation agent that watches, understands, and persuades.

Aug 2 – Aug 11

You're registered

Make a submission →

Overview | Problem statement | My submission | Timeline | Prizes

SmartReco — Build a Behavioral AI Recommendation Agent

The Challenge

Build a web platform (think of a course/product site like an online learning marketplace) that watches how each user behaves and intelligently recommends the right products to them — with AI-generated, persuasive messaging that actually motivates them to take action.

This is not a simple "related products" widget. You are building an **agentic recommendation system**: a backend agent that continuously observes a user's activity, understands their interests, retrieves the most relevant products, and generates personalized, convincing recommendations that update as the user's behavior changes.

The Story (what you're building, end to end)

A user lands on your platform and starts exploring — browsing products, searching, clicking around. Every meaningful action is tracked. Behind the scenes, an AI agent watches this activity build up: *"this user keeps landing on agentic-AI content, searched for it twice, spent time on the advanced courses."*

The agent then reasons over that behavior, retrieves the most relevant products from a knowledge base, and generates a personalized recommendation — not a bare list, but a compelling message: a short narrative about why this matters to them, followed by the specific courses/products that fit their journey.

These recommendations are stored and shown to the user on the site, and they refresh as the user's behavior evolves. As a highlighted bonus, they can even be delivered proactively — e.g. an email in the afternoon recapping the morning's interests with a persuasive story and tailored recommendations.

What You Will Build

1. The Platform (foundation)

the product catalog.

- A clean database schema with the tables your system needs — users, products, activity/events, and stored recommendations — properly related.

2. Product Management with Dual-Write

- An admin can add, edit, and delete products/courses (title, description, category, price, etc.).
- Critically: when a product is added, it must be written to **both** your main database **and** a vector database (so it can be retrieved semantically later). The two stores must stay in sync as products change.

3. Behavioral Event Tracking (a core focus)

- Track meaningful user activity on the frontend: page/product views, searches, clicks, time spent, etc.
- Tracking must be **efficient and non-blocking** — it must not slow down or break the frontend. Think about batching, throttling high-frequency events, and sending data without freezing the user experience.
- Store the events in the backend with a sensible schema (who, what, when).

4. The Agentic Recommendation Engine (the heart of it)

- A backend agent that consumes a user's tracked activity, reasons about their interests, and decides what to recommend.
- Use **RAG / semantic retrieval** over your vector database to fetch the products most relevant to that user's behavior — recommendations must be grounded in your real catalog, not made up.
- Generate a personalized, **persuasive** recommendation — a short convincing narrative plus the specific recommended products. The persuasion should reflect this user's

5. Efficiency & Production Thinking (judged)

- Be smart about **when and how often you call the AI** — don't fire an LLM call on every single user action. Use meaningful triggers and caching to avoid wasteful, redundant calls.
- Be smart about how you store events — efficient, batched, non-blocking.

Highlighted Bonus (good to have — these make you stand out)

These are optional but strongly encouraged. They separate a solid submission from an exceptional one:

- ⭐ **Structured agent framework:** build the agent as an explicit reasoning workflow (e.g. with LangGraph) — nodes that analyze the query/activity, decide when to retrieve, evaluate retrieval quality, refine, and generate.
- ⭐ **Scheduled proactive delivery:** send personalized recommendations by email or Telegram at a set time (e.g. a daily digest based on the day's activity), using a real background scheduler (Celery Beat / APScheduler / cron) — not a manual button.
- ⭐ **Observability:** integrate tracing (e.g. LangSmith) so the agent's workflow is observable end to end.
- ⭐ **Retrieval polish:** smarter retrieval — re-ranking, metadata filtering, better chunking, or a graph-based approach.

Required & Suggested Stack

- **Backend:** Flask or FastAPI (Python) — **required**
- **LLM access:** all LLM/AI calls must go through Mesh API — **mandatory**
- **Vector DB:** any — Chroma, Pinecone, Qdrant, FAISS, or similar

- **Agent (bonus):** LangGraph · **Scheduling (bonus):** Celery / APScheduler
- **Observability (bonus):** LangSmith

Keep your API keys in a local `.env` (do not commit it). Since your code is what's evaluated, your credentials are never needed by us.

Submission Requirements

- A **public GitHub repository** containing all your code (this is what gets evaluated).
- A **README** explaining what you built, the architecture, how to set it up and run it, and which bonus features you implemented.
- (Optional) A short demo video and a deployed URL — reviewed for finalists.

What a Great Submission Looks Like

- Tracking that captures rich behavioral signals without slowing the site down.
- Products genuinely dual-written to SQL and a vector database, kept in sync.
- An agent that actually uses the user's behavior to drive catalog-grounded recommendations — not generic popular-product lists.
- Persuasive copy that reflects the specific user's interests.
- Production thinking: efficient AI-call triggering, caching, batched events, and (bonus) proper scheduled delivery.

Build something you'd actually want running on a real learning platform. 🚀

Using Mesh API (required)

more).

Create an API key (starts with `rsk_`) on the Mesh dashboard, then point the OpenAI SDK at Mesh:

```
from openai import OpenAI
client = OpenAI(base_url="https://api.meshapi.ai/v1", api_key="rsk_...")
client.chat.completions.create(
model="openai/gpt-4o",
messages=[{"role": "user", "content": "Hello"}],
)
```

Add your key to a gitignored `.env` and as the GitHub secret `MESH_API_KEY`. Full guide: Mesh API docs.

Setup & Requirements

Follow this to make sure the automated checks pass and your project is eligible for evaluation.

Your repository must contain:

- All your source code.
- A `requirements.txt` (or `pyproject.toml` / `Pipfile`) listing a web framework (`flask` or `fastapi`) and the LLM client you use through Mesh API.
- A `README.md` describing what you built, your architecture, and how to run it.
- A `.gitignore` that includes `.env` — never commit secrets.

- `MESH_API_KEY` — your Mesh API key. **Mandatory** — every LLM/AI call must go through Mesh API.
- `SUBMISSION_TOKEN` — your submission token, shown on your dashboard as soon as you register.

Your keys stay in your own repository and are only used by checks running in your own GitHub Actions — they are never sent to us.

Enabling the automated checks:

1. **Download the workflow file:** https://careerapi-production.krishnaik.in/api/ci/hackathons/smartreco-build-challenge-2026/workflow.yml
2. Add it to your repository at `.github/workflows/smartreco-checks.yml` (create the `.github/workflows/` folders if they don't exist).
3. Add the secrets listed above (see the secrets guide).
4. Push a commit. The checks run automatically — results appear in your repository's **Actions** tab and on your submission dashboard.

Critical checks (must pass for your commit to be eligible for evaluation):

- **Code compiles** — all Python files are free of syntax errors.
- **Dependencies present** — your requirements list a web framework and an LLM client.

Advisory checks (feedback only — these do not block you): no committed `.env`, a README is present, a `.gitignore` that ignores `.env`.

The automated checks are a first filter — passing them does not finalize your score. A failing critical check simply means "fix and push again"; it is not a penalty.

Mesh API Docs (required) | Flask Documentation | FastAPI Documentation | LangGraph (bonus)

Job Prep
BY KRISHNAIK